

数值相对论与 AMSS-NCKU：模拟黑洞并合

目 录

1 引言：为什么需要数值相对论	2
2 什么是数值相对论	2
2.1 广义相对论与场方程	2
2.2 为什么必须数值求解	2
3 BSSN 形式	3
3.1 为什么不直接演化原始方程	3
3.2 BSSN 分裂与共形变量	3
3.3 时间积分与 Runge-Kutta	3
4 自适应网格细化 (AMR)	4
4.1 为什么要细化网格	4
4.2 AMR 的层级结构	4
5 双黑洞并合的三个阶段	5
5.1 引力波提取	5
6 AMSS-NCKU 程序结构	5
7 完整流程：从参数到波形	6
8 测试用例 GW250118	6
9 本实验的关注点	6
10 本章你将学会	6
11 要点速查	7
12 小结	7

1 引言：为什么需要数值相对论

1915 年爱因斯坦提出**广义相对论** (General Relativity)，告诉我们时空会被物质和能量弯曲。一百年后的 2015 年，人类首次直接探测到**引力波** (Gravitational Wave)，那是由两个黑洞并合产生的时空涟漪。要理解这些观测信号，物理学家必须用计算机模拟黑洞并合的完整过程，这正是**数值相对论** (Numerical Relativity) 的核心任务。

本实验使用的 AMSS-NCKU 程序是中国首个自主开发的数值相对论软件，由中国科学院数学与系统科学研究院 (AMSS) 与台湾成功大学 (NCKU) 联合开发。它求解爱因斯坦场方程，模拟双黑洞从螺旋靠近到并合再到振铃的全过程，并提取引力波形供物理分析。

直觉 你不需要懂相对论物理，就像修车师傅不必懂发动机设计图纸。本实验我们只关心程序结构和数据流：哪个模块吃数据，哪个模块吐结果，哪里算得慢，哪里可以优化。物理正确性由原作者保证，我们只做“不改变物理”的工程优化。

本章我们先理解数值相对论解决什么问题，再认识 BSSN 演化方程和自适应网格细化 (AMR)，然后梳理 AMSS-NCKU 的程序结构与数据流，最后用测试用例 GW250118 把全流程串起来。

2 什么是数值相对论

2.1 广义相对论与场方程

广义相对论的核心是**爱因斯坦场方程** (Einstein Field Equations)：

$$G_{\mu\nu} = 8\pi T_{\mu\nu} \quad (2.1)$$

这里 $G_{\mu\nu}$ 是**爱因斯坦张量** (Einstein Tensor)，描述时空的几何弯曲程度； $T_{\mu\nu}$ 是**能动张量** (Stress-Energy Tensor)，描述物质和能量的分布。方程的物理含义可以概括为：物质告诉时空如何弯曲，时空告诉物质如何运动。

对于真空中的黑洞， $T_{\mu\nu} = 0$ ，方程简化为 $G_{\mu\nu} = 0$ 。即便如此，这仍然是一组十个耦合的非线性偏微分方程，在动态双黑洞场景下没有解析解。

2.2 为什么必须数值求解

在少数高度对称的情形下，例如静态的**史瓦西黑洞** (Schwarzschild Black Hole) 和稳态旋转的**克尔黑洞** (Kerr Black Hole)，可以求得解析解。但双黑洞动态并合毫无对称性可言，两个黑洞互相绕转、扭曲时空，解析方法彻底失效。

爱因斯坦本人曾怀疑引力波是否真实存在，更不用说动态双黑洞解了。数值相对论直到 2005 年才实现第一次成功的双黑洞并合模拟，这一突破被称为数值相对论的“周年纪念”。

数值求解的思路是把连续的时空离散成网格，在网格点上逐时间步推进方程。这需要**有限差分法** (Finite Difference Method)，把偏导数变成网格点上的差商。例如一阶空间导数可以近似为 $f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$ ，其中 h 是网格间距，达到二阶精度。AMSS-NCKU 使用四阶中心差分，涉及更多网格点但精度更高。diff_new.f90 就是实现这些差分运算的 kernel。

3 BSSN 形式

3.1 为什么不直接演化原始方程

直接对原始爱因斯坦方程做数值演化会遇到严重的稳定性问题。方程中含有的**约束** (Constraint) 在离散后会不断累积误差，导致数值爆炸。2005 年之前的数十年，无数研究者栽在这个问题上。

直觉 不妨把约束想象成一根绷紧的橡皮筋：理论上它应该始终保持某根长度不变，但数值误差会让它越拉越长，最后“啪”地断掉，整个模拟崩溃。BSSN 形式就是换一种系橡皮筋的方式，让它不容易断。

3.2 BSSN 分裂与共形变量

BSSN (Baumgarte-Shapiro-Shibata-Nakamura 形式) 通过对原始变量做一系列改写，把方程变成适合数值演化的形式。其核心思想包括：

1. 把四维时空分裂为时间加三维空间，单独演化空间变量。
2. 引入**共形变量** (Conformal Variables)，把空间度规写成 $\gamma_{ij} = \varphi^4 \hat{\gamma}_{ij}$ ，将发散的尺度吸收到共形因子 φ 。
3. 引入辅助变量 $\tilde{A}_{ij}$ 和 $\tilde{\Theta}$ ，保证约束在数值演化中稳定传播。

BSSN 演化的场变量集合 u 包括：共形因子 φ ，共形度规 $\hat{\gamma}_{ij}$ ，外曲率迹 K ，无迹外曲率 $\tilde{A}_{ij}$ ，以及**共形连接函数** (Conformal Connection Functions) $\hat{\Gamma}^i$ 。每个网格点都要存储这些变量，它们合在一起描述了时空的瞬时状态。

我们不展开物理推导，只需知道：BSSN 是一套“改写后的演化方程”，输入是当前时刻的场变量集合，输出是下一时刻的更新值。AMSS-NCKU 程序里的 `bsn_rhs.f90` 负责计算这些演化方程的右端项 (RHS, Right-Hand Side)，`diff_new.f90` 负责空间差分运算，`rungekutta4` 负责时间积分。

回到我们的问题：BSSN 让数值演化不再爆炸，但它引入了更多变量和更复杂的差分运算，计算量增大，这正是后续需要优化的地方。

BSSN 是数值相对论的事实标准，几乎所有现代数值相对论代码（如 *Einstein Toolkit*、AMSS-NCKU）都采用它。你不必推导它，但要知道它在程序里对应哪些文件。

3.3 时间积分与 Runge-Kutta

BSSN 演化方程可以抽象写成：

$$\frac{\partial u}{\partial t} = L(u) \quad (3.1)$$

其中 u 是所有场变量的集合， L 是空间差分算子。时间推进用**四阶 Runge-Kutta** (Runge-Kutta 4) 方法，每一步做四次中间估计，达到四阶时间精度。

例 取小数值算给你看：设场变量 $u = 1$ ，右端项 $L = 0.1$ （不随 u 变化），时间步 $\Delta t = 0.2$ 。四阶 Runge-Kutta 的四个中间值都等于 0.1（因为 L 不依赖 u ），最终更新：

$$u \rightarrow 1 + \left(\Delta \frac{t}{6}\right)(k_1 + 2k_2 + 2k_3 + k_4) = 1 + \left(\frac{0.2}{6}\right)(0.6) = 1.02 \quad (3.2)$$

实际程序里的 u 是整个三维网格上的所有场变量， L 涉及复杂的空间差分，但算法结构完全一样，只是从标量运算变成了整个网格的并行运算。

4 自适应网格细化 (AMR)

4.1 为什么要细化网格

黑洞附近的时空弯曲最剧烈，方程的解变化最快，需要很高的空间分辨率才能算准；而远离黑洞的地方弯曲平缓，粗网格就够。如果全空间都用细网格，计算量爆炸；如果全用粗网格，物理精度不够，黑洞附近会算出错误结果。

直觉 不妨用拍照来类比：你想拍清人脸细节，人脸区域要高像素；但远处的山脉不需要那么清晰，低像素足够。AMR 就是“人脸区域细，背景粗”的自适应策略，在保证精度的同时节省大量算力。

4.2 AMR 的层级结构

自适应网格细化 (Adaptive Mesh Refinement, AMR) 把计算区域组织成多层网格。最外层是覆盖整个计算区域的粗网格，称为**第 0 层** (Level 0)；在黑洞附近加密一层，得到**第 1 层** (Level 1)，网格间距减半；再加密，得到**第 2 层** (Level 2)，间距再减半，依此类推。

每层网格被分成若干**块** (Patch)，每个 patch 是一个独立的小长方体网格。不同层之间和同层不同 patch 之间需要交换数据：

1. **Prolongation (延拓)**：粗网格数据插值到细网格边界，作为细网格的边界条件。
2. **Restriction (限制)**：细网格计算结果取平均后回填到粗网格，校正粗网格值。
3. **Ghost Exchange (鬼单元交换)**：同层相邻 patch 之间交换边界“鬼单元”数据，保证差分 stencil 跨块正确。

这些操作每次时间推进都要执行，是 AMR 的主要通信开销，也是优化的重点区域之一。

AMR 的时间推进采用 **Berger-Oliger 算法** (Berger-Oliger Algorithm)：粗网格走一个大时间步 Δt 时，细网格走多个小时间步（间距为 $\Delta \frac{t}{2}$ ），在粗细层之间通过 prolongation 和 restriction 同步数据。这使得细网格区域的时间精度与空间精度匹配，同时保持整体时间推进的协调性。

例 算一个三层 AMR 网络的总点数。设 AMSS-NCKU 采用等点数加密策略，每层网格都是 64^3 个点，共 3 层。总点数为：

$$3 \times 64^3 = 3 \times 262144 = 786432 \quad (4.1)$$

如果不用 AMR，把全空间都用最细层（第 2 层）的分辨率，由于每加密一层间距减半，第 2 层间距是第 0 层的 $\frac{1}{4}$ ，覆盖同样区域需要 $256^3 = 16777216$ 个点，是 AMR 方案的约 21 倍。这就是 AMR 节省算力的关键所在。

5 双黑洞并合的三个阶段

双黑洞并合是一个动态过程，分为三个特征阶段，每个阶段的物理特征和数值需求不同：

- 1. **Inspiral（旋进）**：两个黑洞因引力波辐射损失能量，沿螺旋轨道互相靠近。这个阶段持续最长，演化步数最多，是计算的主要开销。
- 2. **Merger（并合）**：两个黑洞的事件视界融合为一个，时空剧烈变化，需要最高分辨率捕捉物理细节。
- 3. **Ringdown（振铃）**：并合后的黑洞通过辐射引力波逐步衰减到稳态克尔黑洞，信号呈阻尼振荡。

数值相对论的目标是把这三个阶段的时空演化算出来，再从中提取引力波形，与 LIGO 等探测器的实际观测信号对比验证。

5.1 引力波提取

演化过程中，程序在每个时间步从外层网格提取**纽曼-彭罗斯标量**（Newman-Penrose Scalar） ψ_4 ，它在外层球面上计算，与引力波的两个极化模式直接相关。 ψ_4 的时间序列就是模拟得到的引力波形，存入 `bssn_psi4.dat` 文件，可以与 LIGO 等探测器的实际信号对比验证。此外，`bssn_BH.dat` 记录黑洞位置随时间的演化，`bssn_constraint.dat` 记录约束 violation 以监控数值稳定性。

6 AMSS-NCKU 程序结构

AMSS-NCKU 是一个多语言混合程序，每种语言负责最适合它的工作：

语言	职责	典型文件
C++	网格管理, patch 划分, MPI 通信, 整体控制流	AMSS_NCKU.C, Block.hpp
Fortran	CPU 数值 kernel, 计算演化方程右端项和差分	bssn_rhs.f90, diff_new.f90, rungekutta4
CUDA	GPU kernel, 把 Fortran kernel 移植到 GPU 并行	*_gpu.cu
Python	输入参数生成, 运行 driver, 后处理与绘图	AMSS_NCKU_Input.py, AMSS_NCKU_Program.py

直觉 不妨把程序想象成一支团队：Python 是“项目经理”，负责读需求、下发任务、整理报告；C++ 是“工头”，管理工地（网格）、调度工人（MPI 进程）、搬运材料；Fortran 是“技术工人”，做具体的数值计算活；CUDA 是“技术工人的 GPU 版本”，同样的活用 GPU 并行加速。

这种分层设计的好处是职责清晰：优化数值 kernel 时主要看 Fortran 和 CUDA 文件，优化通信时主要看 C++ 文件，调整运行参数时改 Python 文件。对优化者来说，理解每种语言的职责边界，就能快速定位性能瓶颈所在的代码区域。

7 完整流程：从参数到波形

AMSS-NCKU 的完整运行流程是一条流水线，每个阶段的输出是下一阶段的输入：

1. `AMSS_NCKU_Input.py`: 用户在此设置物理参数（黑洞质量、自旋、初始间距）和计算参数（网格层数、MPI 进程数、是否用 GPU）。
2. `AMSS_NCKU_Program.py`: driver 脚本，读取输入参数，生成 parfile（参数文件）传递给后续可执行文件。
3. `TwoPunctureABE`: 求解初始时刻的两个黑洞度规，生成初值数据文件，为演化提供起点。
4. `ABE` 或 `ABEGPU`: 主演化程序，读入初值，按 BSSN 方程逐步推进时空，在每个时间步提取引力波信号和约束量。
5. `binary_output`: 整理演化结果，输出数据文件供绘图检查。

选择 ABE 还是 ABEGPU 取决于输入参数 `GPU_Calculation`: 设为 "no" 用 CPU 版本 ABE，设为 "yes" 用 GPU 版本 ABEGPU。下一章将详细讲解构建与运行的具体操作。

8 测试用例 GW250118

本实验的测试用例 **GW250118** 基于一个真实的引力波事件：主黑洞质量约 10.3 倍太阳质量，次黑洞约 6.9 倍太阳质量，信噪比 SNR 为 10.5，距离地球约 940 Mpc（兆秒差距）。

课程版本做了适当裁剪，计算规模和演化时间都缩减到可在实验时间内完成。程序内部使用无量纲变量（Dimensionless Variables），即把所有物理量除以适当的基本单位（如太阳质量、引力常数 G 和光速 c 构成的单位制），使得数值大小在 $O(1)$ 量级，便于计算且避免浮点精度问题。

无量纲化是计算物理的标准做法：把 10.3 倍太阳质量变成 1.0（以 10.3 倍太阳质量为单位），避免大数和小数混算时的浮点精度问题。

9 本实验的关注点

本实验的核心目标是**端到端优化**（End-to-End Optimization）一条科学计算流水线，而不是孤立地优化某个 kernel。这意味着：

1. 我们关注从初值生成到演化输出整个流程的耗时分解，找出最耗时的阶段。
2. 优化可以涉及编译选项、运行参数、通信模式、kernel 实现等多个层面。
3. 优化的底线是不改变物理结果，正确性用 `check.sh` 脚本验证。

后续章节将分别讨论构建运行、性能分析、优化策略等主题，逐步展开如何在不改变物理的前提下提升 AMSS-NCKU 的运行速度。

10 本章你将学会

1. 解释数值相对论解决什么问题，为什么爱因斯坦场方程在动态场景下必须数值求解。
2. 描述 BSSN 形式的作用（把方程改写成数值稳定的形式），指出它在程序中对应的文件（`bssn_rhs.f90`、`diff_new.f90`、`rungekutta4`）。
3. 说明 AMR 为何必要，解释 `prolongation`、`restriction`、`ghost exchange` 三种操作的含义。

4. 画出 AMSS-NCKU 的程序结构（四种语言的职责）和数据流（从输入参数到引力波形的完整流程）。
5. 区分 TwoPunctureABE（初值生成）、ABE（CPU 演化）、ABEGPU（GPU 演化）三个可执行文件的职责。

11 要点速查

概念	要点
数值相对论	用计算机数值求解爱因斯坦场方程, 模拟黑洞并合
爱因斯坦场方程	$G_{\mu\nu} = 8\pi T_{\mu\nu}$, 真空时 $T_{\mu\nu} = 0$
有限差分法	偏导数近似为网格点差商, 四阶中心差分
BSSN 形式	改写方程使其数值稳定, 对应 <code>bssn_rhs.f90</code>
时间积分	四阶 Runge-Kutta, 对应 <code>rungekutta4</code>
AMR	黑洞附近细网格, 远处粗网格, 逐层加密
AMR 通信	Prolongation, Restriction, Ghost Exchange
Berger-Oliger	粗网格大步, 细网格小步, 层间同步
程序语言	C++ 控制, Fortran CPU kernel, CUDA GPU kernel, Python 驱动
三个可执行文件	TwoPunctureABE 初值, ABE CPU 演化, ABEGPU GPU 演化
测试用例	GW250118, 真实引力波事件, 无量纲变量
优化底线	不改变物理结果, 用 <code>check.sh</code> 验证

12 小结

本章从“为什么需要数值相对论”出发，介绍了爱因斯坦场方程为何在动态双黑洞场景下必须数值求解，BSSN 形式如何把方程改写成数值稳定的形式，AMR 如何在保证精度的同时大幅节省算力。我们梳理了 AMSS-NCKU 程序的多语言结构和完整数据流，并用测试用例 GW250118 把全流程串起来。本实验的核心目标是端到端优化这条科学计算流水线，后续章节将逐步展开构建、运行、分析与优化的具体操作。

讲义基于 HPC101 2025 Lab4 实验内容编写